

SAARA KETHA HARVEST HUB · ISE_WE_0101_23

Epic 4 – Payments, Analytics & Reporting

Concepts, validations, demo script and viva questions

1. Epic overview
2. Core concepts
3. Validations
4. Suggested live demo
5. Likely viva questions and model answers
6. Know your limitations
7. Where to find it in the code
8. Appendix – system foundations (same in every guide)

Study guide generated from the implemented system · October 2026

1. Epic overview

Epic 4 handles the **money and the insight**: calculating what each farmer is paid from the graded quantity, applying authorised deductions, approvals and payouts, recording buyer payments against invoices, issuing receipts, tracking outstanding amounts, and giving the Manager reports, performance analysis, forecasts and exports.

Stakeholders: Finance Officer, Farmer (payment transparency), Manager (management), Payment service providers (secondary).

1.1 Story map

Story	What happens	Screen	API	DB
E4-US1 S1	Calculate farmer payment from accepted quantity × grade price	CalculatePayment , PendingPayments	POST /api/farmer-payments/calculate	farmer_payments, farmer_invoices
E4-US2 S1	Authorised deductions, approval, rejection, payout	ApprovePayments	PATCH /:id/approve , /reject , /mark-paid	farmer_payment_deductions, pay_farmer_payment()
E4-US3 S1	Farmer views payments and status, prints receipt	MyPayments	GET /api/farmer-payments , /:id/receipt	farmer_payments, payment_receipts
E4-US4 S1	Buyer payments (partial allowed) and invoices	BuyerInvoicesFinance , BuyerInvoices	POST /api/invoices/:id/payment	record_buyer_payment() , invoices, buyer_payments
E4-US5	Payment receipts and outstanding payments	OutstandingPayments , PaymentHistory	GET /api/invoices/outstanding	payment_receipts, view outstanding_payments
E4-US6	Collection, inventory and quality reports	ManagementReports (Operations)	GET /api/reports/management	report_operations_overview()

Story	What happens	Screen	API	DB
E4-US7	Farmer, buyer and waste performance	ManagementReports (Performance)	same call	<code>report_farmer_performance()</code> , <code>report_buyer_performance()</code> , <code>report_waste_analysis()</code>
E4-US8	Revenue, expenses, profit estimate, supply, demand, price trends	ManagementReports (Financial / Market)	same call	<code>report_financial_summary()</code> , <code>report_supply_demand()</code> , <code>report_price_trends()</code>
E4-US9	Forecasts and PDF / CSV export	ManagementReports	GET <code>/api/reports/export/:type</code>	least-squares forecast in <code>report_supply_demand()</code>

2. Core concepts

2.1 How a farmer payment is calculated (E4-US1)

```
gross    = accepted_qty_kg × price_per_kg          (price from the ACTIVE price list for crop + grade + centre)
net      = gross - total_deductions
Example: 118.5 kg × LKR 160 = LKR 18,960.00 gross; deduction "transport LKR 500" → net LKR 18,460.00
```

- The inputs come from the **approved inspection** (accepted quantity and grade) – the finance officer cannot type a quantity. This links quality to money.
- The price comes from `get_current_price()` (the active price for that crop and grade, set by an administrator or finance officer in Price Management). A finance officer may override the price only with a **written justification note**.
- Calculating again is allowed only while the payment is still *calculated* (not yet approved or paid).

2.2 Payment life-cycle and separation of duties

```
calculated —approve→ approved —pay→ paid (receipt issued)
      |
      |—reject (reason)→ rejected
```

- **Maker-checker (separation of duties):** a system setting `require_payment_approval_separation` can force that the person who calculated a payment cannot also approve it (403). Administrators are exempt.
- **Payout needs approval:** `pay_farmer_payment()` refuses to pay anything that is not *approved* or is already paid.
- When paid, a numbered **payout receipt** is created (`RCT-...`); the farmer can print it.

2.3 Buyer payments, partial payments and receipts (E4-US4 / US5)

- An **invoice** has total, amount paid and balance. Buyers may pay in **instalments**; each payment creates its own numbered receipt and the invoice becomes *paid* when the balance reaches zero.
- `record_buyer_payment()` works inside one transaction, so the invoice, payment row and receipt always agree.
- Outstanding payments:** receivables (buyers owe us) and payables (we owe farmers) with **days overdue** so finance can chase them.

2.4 Management analytics (E4-US6 – US9)

Area	What it shows	How
Operations	Collections, accepted vs rejected, inventory and quality overview	<code>report_operations_overview()</code> aggregates SQL
Performance	Farmer ranking (volume, acceptance %), buyer behaviour (orders, cancellation %), waste by reason and crop	SQL report functions, filtered by date range
Financial	Revenue, farmer payouts, wastage cost, profit estimate = revenue – payouts – wastage , monthly chart	<code>report_financial_summary()</code>
Supply & demand	Monthly supply (collected kg) vs demand (ordered kg)	<code>report_supply_demand()</code>
Forecast	Next 3 months of supply and demand	Least-squares linear trend over the history; the method is stated on the screen so nobody mistakes it for a black box
Price trends	Average purchase / selling price per crop by month	<code>report_price_trends()</code>
Export	CSV (server generated) and PDF (print)	GET <code>/api/reports/export/:type</code>

Be ready to explain the forecast: a straight line $y = a + b \cdot x$ is fitted through the monthly history by minimising squared errors (least squares); the line is extended three months ahead. It is a simple trend estimate, not a seasonal model, and it is labelled as such.

The **Manager role** is **read-only**: it can see reports and orders but every write endpoint rejects it (403).

3. Validations

3.1 Calculating a farmer payment – E4-US1 / US2

Rule	Result
Only the finance officer may calculate	403 for other roles
<code>collection_id</code> required; collection must exist	400 / 404
A completed quality inspection must exist	409 "A completed quality inspection is required before a payment can be calculated"
Accepted quantity must be > 0 (a fully rejected delivery has nothing to pay)	409 "Nothing was accepted for this collection..."

Rule	Result
Cannot recalculate once approved or paid	409 "Payment PAY-... is already approved and can no longer be recalculated"
A purchase price must exist for that crop and grade (or be supplied as an override)	409 "No active purchase price is configured... set one in Price Management"
A price override requires a note	400 "A note explaining the price override is required"
<code>deductions</code> must be a list; each needs a description and an amount > 0	400
Total deductions cannot exceed the gross amount	400 "Deductions (LKR x) cannot exceed the gross amount (LKR y)"
Money is rounded to 2 decimals; payment number is unique (retry on collision)	–

3.2 Approving, rejecting, paying – E4-US2

Rule	Result
Approve only if status is <i>calculated</i> / pending approval	409 "...only calculated payments can be approved"
Maker ≠ checker (when the setting is on)	403 "A payment must be approved by someone other than the person who calculated it"
Reject needs a reason of at least 5 characters	400
Cannot reject something already paid or rejected	409
Pay only if approved ; cannot pay twice	INVALID_STATE → 409 "payment is already paid" / "must be approved before it can be paid"
All finance actions restricted to <code>finance_officer</code>	403 otherwise

3.3 Farmer sees payments – E4-US3

- Roles allowed: farmer, finance officer, manager.
- **Ownership:** a farmer opens only their own payment or receipt (403 otherwise); the list is filtered to their farmer id.
- The payout receipt exists only after payment ("A receipt is issued once the payment has been paid" – 404 before that).

3.4 Buyer payments – E4-US4 / US5 (`record_buyer_payment()`)

Rule	Error
Invoice must exist	NOT_FOUND → 404
Invoice must not be cancelled	INVALID_STATE → 409
No duplicate payment of a fully paid invoice	INVALID_STATE → 409 "invoice is already fully paid"
Amount must be > 0	INVALID_QTY → 400

Rule	Error
Order must be approved first (not submitted, under review, rejected, cancelled or draft)	INVALID_STATE → 409
No overpayment: amount ≤ outstanding balance	INVALID_QTY → 400 "payment of x exceeds outstanding balance of y"
Only the finance officer records payments	403

3.5 Management reports and export – E4-US6 to US9

- Report and export endpoints allow **manager** and **finance officer** (inventory manager only for exports); other roles → 403.
- Date range `from / to` is parsed with defaults; the export `type` must be one of the supported report names (otherwise 404 "Unknown report type").
- **CSV formula-injection protection:** any cell that starts with `=`, `+`, `-` or `@` is prefixed so Excel cannot execute it as a formula; commas and quotes are escaped. This is covered by an automated test.
- PDF export uses the browser print engine on the on-screen report, so what you see is what is exported.

3.6 Price management (feeds payments)

- Only the **administrator or finance officer** creates or changes prices; the active price for a crop/grade/centre is resolved by `get_current_price()`.
- A price has a status (active / inactive / superseded) so history stays for the price-trend report.

4. Suggested live demo

1. Finance officer → **Calculate payment** for an inspected collection: show quantity and grade price pulled automatically. Add a deduction larger than the gross → refused; then a valid "transport" deduction (US1, US2).
2. **Approve & disburse:** approve, then mark paid; try paying an unapproved payment to show the rule.
3. Farmer → **My payments:** statuses and the printable payout receipt (US3).
4. Finance → **Buyer invoices:** record a partial payment, show the receipt and the balance; then try to overpay → refused; pay the balance → invoice becomes paid (US4).
5. **Outstanding payments:** receivables, payables and days overdue (US5).
6. Manager login → **Management reports:** Operations, Performance (farmer / buyer / waste), Financial with profit estimate, supply-demand forecast and price trends (US6–US8). Show there are no write buttons.
7. **Export CSV** and **PDF** (US9).

5. Likely viva questions and model answers

Q1. How is the farmer payment calculated?

Accepted quantity from the approved inspection × current purchase price for the crop and grade = gross; minus authorised deductions = net. The finance officer cannot alter the quantity; a price override needs a written note.

Q2. What stops finance from paying the wrong amount?

Quantity and grade come from the inspection, price from the price list, deductions are validated (positive, described, not more than gross), and the amount is rounded to 2 decimals and stored on the payment.

Q3. What is separation of duties and do you have it?

The person who prepares a payment should not approve it. A system setting can enforce that the approver differs from the calculator (403 otherwise).

Q4. Why must a payment be approved before payout?

Approval is a control point. The payout database function refuses anything not approved and refuses a second payment, so double payment is impossible.

Q5. How do partial buyer payments work?

Each payment reduces the invoice balance and creates its own receipt. The function rejects amounts above the balance and payments on paid or cancelled invoices; when balance reaches zero the invoice becomes paid.

Q6. Why are payments done in database functions?

Invoice update, payment row and receipt must succeed or fail together; a transaction guarantees consistent financial records.

Q7. What are outstanding payments?

Receivables = unpaid invoices from buyers, payables = approved farmer payments not yet paid, each with days overdue.

Q8. How is the profit estimate computed?

Revenue from buyer sales minus farmer payouts minus wastage cost. It is an estimate because other operating costs are not recorded.

Q9. How does the forecast work?

A least-squares straight line is fitted to the monthly supply and demand history and extended for three months. The screen states the method.

Q10. What can the Manager do?

View reports and orders and export them. The role is read-only, enforced on every write endpoint with 403.

Q11. How do you protect exported CSV files?

Cells starting with = + - or @ are neutralised (formula-injection protection) and the data is escaped; export is limited to authorised roles.

Q12. How does a farmer know their payment status?

My payments lists calculated / approved / paid / rejected with amounts and a printable receipt once paid.

Q13. Where do prices come from?

The administrator or finance officer maintains the price list per crop, grade and centre; payments and the marketplace read the active price.

6. Know your limitations

- No real payment-gateway or bank integration: payments are **recorded** by finance staff after money moves outside the system (the Sprint 0 budget includes a payment module as a future step).
- The profit estimate ignores overheads (salaries, transport, rent) because they are not captured.

- Forecasting is a linear trend, not seasonal or machine-learning based.
- Email notifications of payments depend on an SMTP server being configured; otherwise only in-app notifications are sent.
- PDF export relies on the browser print dialog instead of a server-side PDF generator.

7. Where to find it in the code

Concern	File
Farmer payments	<code>backend/src/controllers/farmerPaymentController.ts</code>
Pay-out, buyer payments, receipts SQL	<code>supabase/migrations/025_receipts_inspection_payments.sql</code>
Invoices and outstanding	<code>backend/src/routes/invoiceRoutes.ts</code>
Management reports and CSV export	<code>backend/src/controllers/managementReportController.ts</code> , <code>supabase/migrations/026_management_reports_and_bid_results.sql</code>
CSV safety test	<code>backend/src/__tests__/csvAndErrors.test.ts</code>
SQL tests	<code>supabase/tests/02_payments_receipts.sql</code>
Frontend	<code>frontend/src/pages/finance/*</code> , <code>pages/farmer/MyPayments.tsx</code> , <code>pages/admin/ManagementReports.tsx</code>

8. Appendix – system foundations (same in every guide)

8.1 Architecture in one picture

```

Browser (React 18 + Vite + Tailwind + TanStack Query + react-hook-form + Zod)
  |  HTTPS, JSON, header  Authorization: Bearer <JWT>
  v
Express REST API (Node.js + TypeScript)
  helmet · CORS · rate limit · authenticate · requireRole · validate(Zod) · controller ·
  errorHandler
  |  supabase-js (service-role key, server side only)
  v
Supabase: PostgreSQL (tables, constraints, RPC functions, triggers) · Auth · Storage (private
buckets)

```

Layered design: presentation (React pages) → API/business layer (Express controllers and services) → data layer (Supabase/PostgreSQL). Business rules that must never be bypassed (stock, weights, payments) are enforced in the **database** through PostgreSQL functions, so even a buggy client cannot break them.

8.2 Authentication vs authorisation

Concept	What it answers	How HarvestHub does it
Authentication	"Who are you?"	Email + password are checked by Supabase Auth . On success the API returns a JWT access token and a refresh token. The frontend stores them and the Axios interceptor adds <code>Authorization: Bearer <token></code> to every request; on a 401 it tries the refresh token once, otherwise logs the user out.
Authorisation (RBAC)	"What may you do?"	<code>authenticate</code> middleware verifies the token with Supabase, loads the user's <code>profiles</code> row (role, account_status) and rejects suspended / inactive / pending accounts. <code>requireRole(...)</code> then allows only listed roles (administrator is a superuser). Wrong role → 403 .
Ownership checks	"Is this YOUR record?"	Role alone is not enough: a farmer must not read another farmer's data. Controllers call helpers such as <code>assertFarmerAccess</code> / <code>assertBuyerAccess</code> (services/access.ts) → 403 if the record belongs to someone else.
Row Level Security (RLS)	DB-level safety net	Policies are enabled on tables in Supabase. The API itself uses the service-role key, so the API-layer checks above are the primary control ; RLS protects direct database access.

The nine roles are: **farmer, collection_centre_officer, quality_inspector, inventory_manager, buyer, finance_officer, transport_coordinator, manager, administrator**. Only **farmer** and **buyer** can self-register; staff accounts are created by an administrator.

8.3 Three layers of validation

Layer	Tool	Purpose	Failure looks like
1. Frontend	Zod schema + react-hook-form	Instant feedback, fewer bad requests. Never trusted on its own.	Red message under the field
2. API	Zod schema via <code>validate()</code> middleware + controller rules	The real gate: format, required fields, ownership, state rules.	HTTP 400 with <code>{success:false, message, errors: [{field,message}]}</code>
3. Database	NOT NULL, UNIQUE, CHECK, enums, foreign keys, PL/pgSQL functions	Guarantees integrity even if layers 1–2 are bypassed.	Exception mapped to 404 / 400 / 409

Say this in the evaluation: "We validate on the client for usability, on the server for security, and in the database for integrity. The client is never trusted."

8.4 HTTP status codes used

Code	Meaning	Example in this system
200 / 201	OK / Created	Profile loaded; account or farmer created
400	Bad request – invalid input	Bad NIC format, weak password, quantity ≤ 0
401	Not authenticated	Missing / expired token, wrong password

Code	Meaning	Example in this system
403	Authenticated but not allowed	Wrong role, or a farmer opening another farmer's record, pending farmer logging in
404	Not found	Unknown farmer / order id
409	Conflict with current state	Duplicate NIC, duplicate booking, invalid status transition, insufficient stock
429	Too many requests	Rate limiter (e.g. crop advisor: 30 requests / 15 min)
500	Server error	Unexpected failure (logged by winston)

8.5 Cross-cutting security measures

- **helmet** security headers; **CORS** restricted to the configured frontend origin in production.
- **Rate limiting** with express-rate-limit (general API and a stricter limit on `/api/auth`).
- **Safe search**: user search text is stripped of characters that could alter a PostgreSQL filter (`, ( ) % * \`) and limited to 60 characters.
- **Mass-assignment protection**: update endpoints copy only a whitelist of columns (a farmer cannot set their own verification status).
- **Generic login error** ("Invalid email or password") so attackers cannot discover which emails exist.
- **Secrets** only in `.env` (service-role key never reaches the browser); private storage buckets with short-lived signed URLs for documents.
- **Notifications and audit**: important actions create in-app notification rows (and emails when SMTP is configured); auction awards and rejections are written to the audit log.